CHAPITRE 6

Les Exceptions Java

Comprendre, Gérer et Maîtriser les Erreurs

CHAPITRE 6

Introduction : Qu'est-ce qu'une exception ?

Définition

Une exception est un événement **anormal** qui survient durant l'exécution d'un programme et qui **interrompt le flux normal** des instructions.

DEUX STRATÉGIES DE GESTION

Gérer l'exception (Handle)
Utiliser un bloc `try-catch` pour résoudre le problème et continuer l'exécution.

Déclarer l'exception (Declare)
Utiliser `throws` dans la signature de la méthode pour déléguer la responsabilité à l'appelant.

Origine : Peut être lancée par la JVM (erreur système) ou par le Programmeur (logique métier).

Main.java

```java
public class Main {
    public static void main(String[] args) {

        // Création d'un tableau vide (taille 0)
        String[] animals = new String[0];

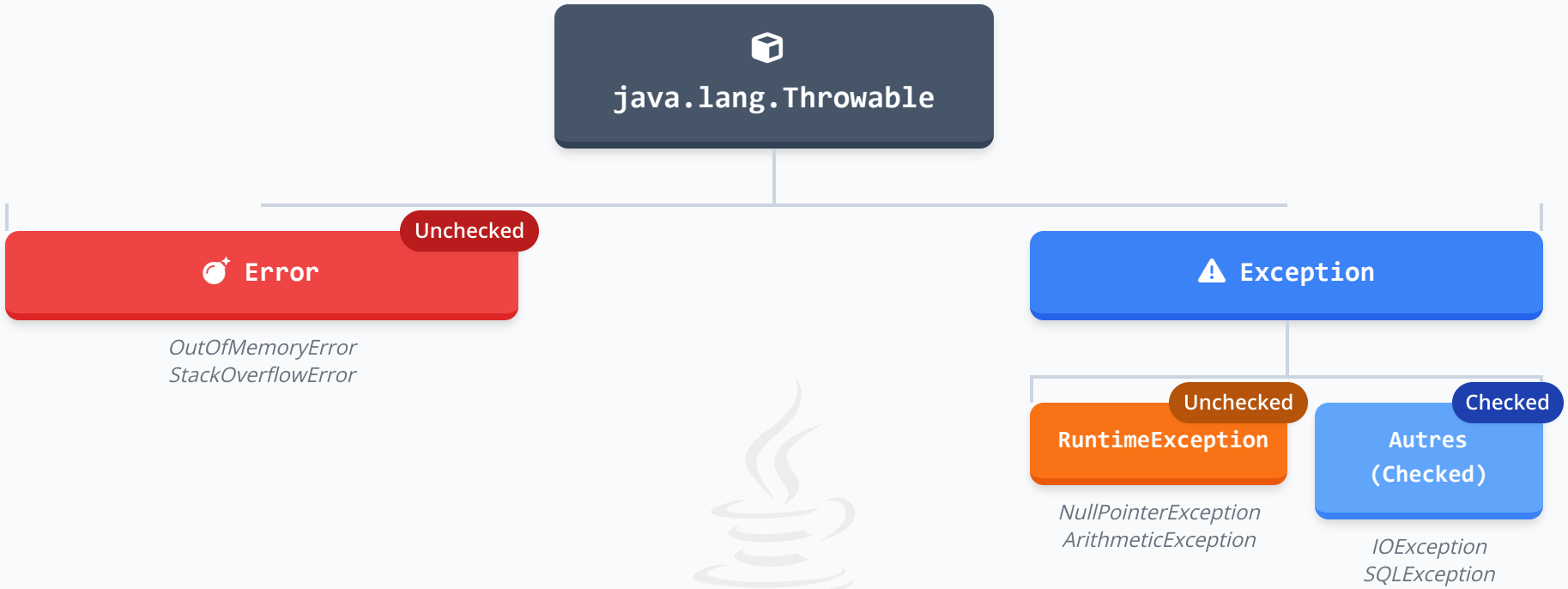
        // Tentative d'accès au premier élément
        // L'index 0 n'existe pas !
        System.out.println(animals[0]);

    }
}
```

RÉSULTAT CONSOLE

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
0
  at Main.main(Main.java:7)
```

Hiérarchie des Exceptions



java.lang.Error

Problèmes graves liés à l'environnement d'exécution (JVM).

Action :
Ne pas essayer de gérer (catch). Le programme ne peut généralement pas récupérer.

RuntimeException

Erreurs de logique de programmation (bug). "Unchecked".

Action :
La gestion est optionnelle. Mieux vaut corriger le code que de catcher l'erreur.

Checked Exception

Événements prévisibles mais hors de contrôle (ex: fichier manquant).

Action :
OBLIGATOIRE. Doit être gérée (try-catch) ou déclarée (throws).

CHAPITRE 6

Exceptions : Vérifiées vs Non Vérifiées

CHECKED Exceptions Vérifiées

- Héritent de **Exception** mais PAS de RuntimeException.
- **Obligation de traitement** (Handle or Declare).
- Prévues par le compilateur (Erreur de compilation si ignorées).
- Exemples : `IOException`, `FileNotFoundException`, `SQLException`.

CheckedExample.java

```java
// DOIT déclarer throws ou utiliser try-catch
void readFile() throws IOException {
    Path p = Paths.get("data.txt");

    // Files.readAllLines lance IOException
    List<String> lines = Files.readAllLines(p);
}
```

⚠ Compilation Error si "throws" absent

UNCHECKED Exceptions Non Vérifiées

- Héritent de **RuntimeException**.
- **Traitement optionnel**. Pas d'obligation de try-catch.
- Indiquent souvent une erreur de logique programmeur.
- Exemples : `NullPointerException`, `ArithmeticException`, `NumberFormatException`.

UncheckedExample.java

```java
void calculateLength(String s) {
    // Pas besoin de déclarer throws

    // Peut lancer NullPointerException
    // si s est null
    int len = s.length();

    System.out.println(len);
}
```

✓ Compile toujours

CHAPITRE 6

Lancer une Exception : Le mot-clé `throw`

Concept

Le mot-clé `throw` permet au programmeur de déclencher **explicitement** une exception.

Cela interrompt immédiatement le flux de la méthode et transfère le contrôle au premier bloc `catch` compatible (ou à l'appelant).

SYNTAXE

```
throw new ExceptionType();
```

ATTENTION

Ne confondez pas **`throw`** (lancer) et **`throws`** (déclarer).

ThrowExample.java

```java
public void registerUser(String username, int age) throws Exception {

// 1. Lancer une Unchecked Exception (RuntimeException)
if (username == null) {
// Pas besoin de déclarer 'throws' pour celle-ci
throw new IllegalArgumentException("Username cannot be null");
}

// 2. Lancer une Checked Exception
if (age < 18) {
// OBLIGATOIRE : doit être déclarée avec 'throws' ci-dessus
throw new Exception("User must be 18 or older");
}

// 3. Propager une exception (Rethrow)
```

Mots-clés Types Strings

CHAPITRE 6

Structure try-catch : Flux d'Exécution

Diagramme de Flux

1. Début du bloc Try

Le code s'exécute normalement ligne par ligne.

2. Exception Levée !

Une erreur survient. Le flux normal est **interrompu** immédiatement.

SAUT

3. Code Ignoré

Le reste du bloc `try` n'est JAMAIS exécuté.

4. Bloc Catch

Java attrape l'exception et exécute le code de gestion.

Correspondance Code

Workflow.java

```java
try {
tomber(); // 1. Exécution normale
throw new Exception(); // 2. BOUM !
System.out.println("Ignoré"); // 3. Sauté
} catch (Exception e) {
seRelever(); // 4. Gestion
}
continuerMarche(); // 5. Reprise
```

Point Clé : Dès qu'une exception est levée, Java abandonne le reste du bloc `try` et cherche immédiatement le `catch` correspondant.

CHAPITRE 6

Le Bloc finally : Nettoyage Garanti

Exécution Systématique

Le bloc `finally` est conçu pour contenir du code qui doit **toujours s'exécuter**, qu'une exception soit levée ou non.

RÈGLES ET USAGES

Libération des Ressources

Fermeture de fichiers, connexions BDD, sockets réseau (cleanup).

Priorité sur le Return

Le bloc `finally` s'exécute même si un `return` est présent dans le `try` ou le `catch`.

Seule exception : System.exit() arrête la JVM immédiatement, sautant le finally.

Zoo.java

```java
public String explorer() {
    try {
        System.out.println("1. Ouvrir le zoo");
        throw new RuntimeException();
    }
    catch (Exception e) {
        System.out.println("2. Attraper l'animal");
        return "Retour anticipé";
    }
    finally {
        // S'exécute malgré le return ligne 8
        System.out.println("3. Fermer la cage");
    }
}
```

CONSOLE OUTPUT

```
1. Ouvrir le zoo
2. Attraper l'animal
3. Fermer la cage (Le finally a bien tourné !)
```

CHAPITRE 6

Capturer Plusieurs Exceptions (Multi-catch)

Concept

Introduit en Java 7, le **multi-catch** permet de capturer plusieurs types d'exceptions dans un **seul bloc catch**. Cela réduit la duplication de code lorsque le traitement d'erreur est identique pour différentes exceptions.

RÈGLES SYNTAXIQUES

Opérateur Pipe (|)

Les types d'exceptions sont séparés par une barre verticale |.

Pas d'Héritage Direct

Interdit de combiner une classe mère et sa fille (ex: `Exception | IOException`) car cela rendrait le code redondant.

Variable Implicitement Finale

La variable d'exception (ex: e) est considérée comme `final` et ne peut pas être réassignée dans le bloc catch.

MultiCatchExample.java

```java
try {
    // Code pouvant lancer plusieurs types d'erreurs
    Files.readAllLines(path);
    database.connect();

} catch (IOException | SQLException e) {

    // Traitement commun pour les deux exceptions
    System.out.println("Erreur système : " + e.getMessage());

    // e = new IOException(); // ERREUR DE COMPILATION !
    // La variable 'e' est effectively final
}
```

⚠ Erreur Commune (Héritage)

```java
try { ... }
catch (FileNotFoundException | IOException e) { }
// ERREUR: FileNotFoundException hérite déjà de IOException.
// Le compilateur considère cela comme du code inaccessible.
```

CHAPITRE 6

Règles : Ordre des Catch & Exceptions Custom

La Règle d'Or

Les blocs `catch` doivent être ordonnés du plus **spécifique** (sous-classe) au plus **général** (super-classe).

⚠ Sinon : Erreur de compilation "Unreachable Code".

Exceptions Custom

Vous pouvez créer vos propres exceptions en étendant les classes de base.

- `extends Exception` = Checked

  `extends RuntimeException` = Unchecked

✕ INCORRECT

Code Inatteignable

La classe parente attrape tout avant la classe enfant.

```
try {
    visiterZoo();
} catch (Exception e) {
    // Attrape TOUT (y compris RuntimeException)
    System.out.println("Erreur générale");
} catch (RuntimeException e) {
    // ERREUR DE COMPILATION !!
    // Ce bloc ne sera jamais atteint
}
```

Unreachable Code

✓ CORRECT

Filtrage Correct

Du plus fin au plus large.

```
try {
    visiterZoo();
} catch (RuntimeException e) {
    // 1. Vérifie d'abord le cas spécifique
    System.out.println("Erreur d'exécution");
} catch (Exception e) {
    // 2. Si ce n'était pas Runtime, attrape le reste
    System.out.println("Autre erreur");
}
```

CHAPITRE 6

Exceptions Runtime Courantes

Types Principaux

Unchecked

ArithmeticException
Division par zéro (entiers uniquement).

ArrayIndexOutOfBoundsException
Accès à un index invalide dans un tableau.

ClassCastException
Tentative de cast vers une sous-classe incompatible.

IllegalArgumentException
Argument passé à une méthode est inapproprié.

NullPointerException (NPE)
Accès à une méthode ou champ sur une référence null.

NumberFormatException
Échec de conversion d'une chaîne en nombre.

CommonErrors.java

```java
public void runtimeExamples() {

    // 1. ArithmeticException
    int x = 10 / 0;

    // 2. NullPointerException
    String s = null;
    s.length();

    // 3. ClassCastException
    Object obj = "Texte";
    Integer i = (Integer) obj;
```

CONSOLE OUTPUT (EXEMPLE NPE)

```
Exception in thread "main" java.lang.NullPointerException
  at CommonErrors.runtimeExamples(CommonErrors.java:6)
```

CHAPITRE 6

Exceptions Vérifiées Courantes (Checked)

Définition

Les exceptions vérifiées (Checked Exceptions) sont des problèmes prévisibles (souvent liés aux entrées/sorties) que le compilateur vous **oblige** à gérer.

EXCEPTIONS FRÉQUENTES

FileNotFoundException

Lancée lorsqu'un fichier spécifié par son chemin est introuvable.

IOException

Superclasse générique pour les problèmes d'Entrée/Sortie (lecture/écriture de fichiers, réseaux).

Règle d'Or : Vous devez soit les capturer avec `try-catch`, soit les déclarer avec `throws`.

Option 1: Déclarer (throws)

```java
import java.io.*;

// La méthode délègue la responsabilité
void lireFichier(String chemin) throws IOException {
    // Code susceptible de lever IOException
    BufferedReader br = new BufferedReader(new FileReader(chemin));
    System.out.println(br.readLine());
    br.close();
}
```

Option 2: Gérer (try-catch)

```java
public static void main(String[] args) {
    try {
        lireFichier("inexistant.txt");
    } catch (IOException e) {
        // Gestion de l'erreur ici
        System.out.println("Erreur I/O : " + e.getMessage());
    }
}
```

SORTIE CONSOLE

```
Erreur I/O : inexistant.txt (The system cannot find the file specified)
```

Errors : Problèmes Graves de la JVM

⚠ Ne pas gérer (Do not handle)

Les **Errors** étendent `java.lang.Error`. Elles indiquent des problèmes sérieux que l'application ne devrait pas essayer de capturer (catch). Elles sont généralement lancées par la JVM.

ExceptionInInitializerError

Lancée lorsqu'une exception survient dans un bloc d'initialisation statique (static initializer) et n'est pas gérée.

StackOverflowError

Lancée lors d'une récursion infinie qui sature la pile d'exécution.

NoClassDefFoundError

La classe était présente à la compilation mais est introuvable à l'exécution (classpath incorrect).

InfiniteRecursion.java

```java
public void doNotCallMe() {
    // Appelle la méthode elle-même sans condition de fin
    doNotCallMe();
}
// Résultat : StackOverflowError
```

StaticFail.java

```java
public class StaticFail {
    static {
        int[] counts = new int[3];
        // Lance ArrayIndexOutOfBoundsException ici
        int num = counts[-1];
    }

    public static void main(String[] args) { }
```

RÉSULTAT CONSOLE (STATICFAIL)

```
Exception in thread "main" java.lang.ExceptionInInitializerError
Caused by: java.lang.ArrayIndexOutOfBoundsException: -1
  at StaticFail.<clinit>(StaticFail.java:4)
```

Méthodes & Héritage : Règles du throws

La Règle de Restriction



Une méthode redéfinie (overridden) dans une sous-classe **ne peut pas** déclarer de nouvelles exceptions vérifiées (checked) ou des exceptions plus larges que la méthode parente.

Pourquoi ?

Polymorphisme : Le code qui utilise la classe parente ne s'attend pas à gérer de nouvelles surprises.

Ce qui est permis

- Déclarer **moins** d'exceptions (ou aucune).
- Déclarer des **sous-classes** plus spécifiques.
- Déclarer n'importe quelle **RuntimeException** (unchecked).

✕ INTERDIT

```
class Hopper {
    public void hop() { }
}

class Bunny extends Hopper {
    // ERREUR DE COMPILATION !
    // Hopper.hop() ne déclare aucune exception checked.
    public void hop() throws Exception { }
}
```

Exception checked ajoutée illégalement

✓ AUTORISÉ

```
class Hopper {
    public void hop() throws IOException { }
}

class Bunny extends Hopper {
    // OK : FileNotFoundException est une sous-classe de IOException
    // OK : RuntimeException est toujours permise (unchecked)
    public void hop() throws FileNotFoundException,
                             RuntimeException { }
}
```

CHAPITRE 6

Conseils pour l'Examen OCA

La Stratégie "Scan"

Face à une question sur les exceptions, suivez cet ordre mental :

1. **Checked ou Unchecked ?**
   Regardez la hiérarchie. RuntimeException = relax. IOException = strict.

2. **Le "Throws" est-il là ?**
   Si une Checked Exception est lancée, la méthode DOIT la déclarer.

3. **Ordre des Catch**
   Vérifiez immédiatement si une super-classe capture l'exception avant sa sous-classe.

Pièges à Éviter

Code Inatteignable (Unreachable)

Un code placé après un `throw` ou un `return` inconditionnel, ou un bloc `catch` inutile (car déjà capturé plus haut).

Override Interdit

Une sous-classe NE PEUT PAS ajouter de nouvelles Checked Exceptions. Elle peut en supprimer ou les rendre plus spécifiques.

Try Orphelin

Un `try` doit avoir SOIT un `catch`, SOIT un `finally` (ou les deux). Un `try` seul ne compile pas.

</> Mots-Clés

`throw`	Action (lancer)
`throws`	Déclaration (méthode)
`finally`	Toujours exécuté

Checklist Finale

- [x] Ai-je vérifié les imports (IOException) ?
- [x] Le bloc finally modifie-t-il le retour ?
- [x] Est-ce une Error (StackOverflow) ?
- [x] Attention aux exceptions masquées (NPE).

Astuce : `main` peut aussi déclarer `throws Exception` !

CHAPITRE 6

Résumé & Checklist : Les Incontournables

Types & Hiérarchie

Checked Exceptions

Héritent de Exception (mais pas RuntimeException).
Obligation : Handle or Declare.

Unchecked Exceptions

Héritent de RuntimeException.
Erreurs de logique. Gestion facultative.

Errors

Héritent de Error. Problèmes JVM graves. Ne pas attraper.

```
Throwable > Exception > RuntimeException
```

Syntaxe & Flux

try-catch-finally

`try` nécessite soit `catch`, soit `finally`. Le code s'arrête net au `throw`.

Throw vs Throws

`throw` (action) lance une instance. `throws` (déclaration) signale un risque.

Bloc Finally

Toujours exécuté (sauf System.exit). Idéal pour fermer les ressources.

Multi-catch

`catch(E1 | E2 e)`. Les types ne doivent pas avoir de lien d'héritage.

Règles & Pièges

Ordre des Catch

Du plus spécifique (enfant) au plus général (parent).
Sinon : erreur de compilation.

Override de Méthode

La sous-classe ne peut PAS ajouter de nouvelles Checked Exceptions. Elle peut en supprimer.

Common Exceptions

- NullPointerException (Runtime)
- ArithmeticException (Runtime)
- IOException (Checked)
- FileNotFoundException (Checked)

"Une exception bien gérée est une opportunité de robustesse."